

MIT 6.86x: 데이터를 조각하는 기술

Unit 2 (Part B): Feature Engineering & Polynomial Features

Contents

1	Unit 2 (Part B). 특징 공학 (Feature Engineering)	2
1.1	1. 비선형성 (Non-linearity)의 필요성	2
1.2	2. 특징 변환 (Feature Mapping)	3
1.3	3. 다항식 특징 (Polynomial Features)	3
1.4	4. 장점과 주의점 (The Trade-off)	4
2	실전 시나리오: 불법 프로그램(해킹) 사용자 탐지	4
3	자주 묻는 질문 (FAQ)	5

Course Structure & Current Focus

- Unit 2 (Part A): Linear Classifiers (직선으로 자르기)
- **Unit 2 (Part B): Feature Engineering (현재 단원: 데이터 차원 확장)**
 - 2.5 Non-linear Separability (직선의 한계)
 - 2.6 Feature Maps $\phi(x)$ (차원 확장)
 - 2.7 Polynomial Features (다항식 변환)
 - 2.8 Curse of Dimensionality (차원의 저주)
- Unit 3: Neural Networks (특징을 스스로 학습하기)

1 Unit 2 (Part B). 특징 공학 (Feature Engineering)

우리는 퍼셉트론과 SVM을 통해 최적의 '직선'을 긋는 법을 배웠습니다. 하지만 세상의 데이터가 모두 직선으로 깔끔하게 나뉘까요? 만약 데이터가 도넛 모양으로 분포한다면 직선 하나로 절대 자를 수 없습니다. 이 한계를 극복하기 위해 우리는 칼(모델)을 바꾸는 대신, 데이터의 모양을 바꾸는 (*Transformation*) 전략을 취합니다.

□ 개요 (Overview)

이 단원은 선형 분리가 불가능한 데이터를 해결하기 위해 입력 데이터 x 를 고차원 특징 공간 $\phi(x)$ 로 매핑하는 방법을 배웁니다. 특히 다항식 특징(Polynomial Features)을 통해 곡선 경계를 만드는 원리와, 차원이 너무 높아질 때 발생하는 계산 비용 및 과적합 문제(Trade-off)를 다룹니다.

□ 핵심 용어 사전

용어 (Term)	직관적 의미 (Meaning)
Non-linear Separability	직선 하나로 100% 분류가 불가능한 상태 (예: XOR, 도넛).
Feature Map (ϕ)	데이터를 저차원에서 고차원으로 옮기는 함수.
Feature Space	매핑된 데이터들이 살고 있는 고차원 세상.
Polynomial Feature	x^2, x_1x_2 처럼 곱하기를 통해 만든 새로운 특징.
Linear in Feature Space	고차원에서는 평면으로 잘리지만, 원래 세상에선 곡선으로 보임.

1.1 1. 비선형성 (Non-linearity)의 필요성

□ 개념 1: 직선의 한계

한 줄 요약: 동그라미 안에 있는 점과 밖에 있는 점은 자 하나로 가를 수 없습니다. 가위(비선형 모델)가 필요하거나, 종이를 접어야(특징 변환) 합니다.

문제 상황: XOR과 도넛

- **XOR:** (0, 0), (1, 1)은 파란색, (0, 1), (1, 0)은 빨간색. 대각선 관계.
- **Donut:** 원점 (0, 0) 근처는 빨간색, 그 바깥 고리 모양은 파란색.

이런 데이터에 퍼셉트론(선형 분류기)을 적용하면, 오차가 줄어들지 않고 계속 진동(Oscillation)하며 실패합니다.

1.2 2. 특징 변환 (Feature Mapping)

□ 개념 2: 차원 띄우기 (Lifting)

한 줄 요약: 2차원 바닥에 흩어진 구슬들을 3차원 공중으로 띄우면, 그 사이로 판(평면)을 끼워 넣을 수 있습니다.

1) 직관적 비유

도넛 모양 데이터를 상상해 봅시다.

- 가운데 빨간 구슬들 (원점 근처)
- 바깥쪽 파란 구슬들 (반지름이 큼)

여기에 새로운 축(고도) $z = x_1^2 + x_2^2$ 를 추가합니다.

- 빨간 구슬: 원점 근처라 z 값이 작음 → 바닥에 깔림.
- 파란 구슬: 원점에서 멀어서 z 값이 큼 → 공중으로 떠오름.
- **해결:** 이제 바닥과 공중 사이에 수평 판(Hyperplane)을 끼워 넣으면 완벽하게 분리됩니다!

2) 수학적 정의

입력 벡터 $x \in \mathbb{R}^d$ 를 고차원 벡터 $\phi(x) \in \mathbb{R}^D$ 로 변환합니다 ($D > d$).

$$\theta \cdot x + \theta_0 \xrightarrow{\text{Transformation}} \theta' \cdot \phi(x) + \theta'_0$$

1.3 3. 다항식 특징 (Polynomial Features)

□ 개념 3: 곱셈으로 특징 만들기

한 줄 요약: 원래 변수들을 서로 곱해서 (x_1^2, x_1x_2) 새로운 정보를 만들어냅니다.

1) 2차 다항식 (Quadratic Features)

입력 $x = [x_1, x_2]^T$ 일 때, 2차항까지 모두 만들면:

$$\phi(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1x_2]^T$$

- 원래 차원: 2차원
- 확장된 차원: 6차원

2) 기하학적 의미

확장된 공간에서의 선형 결함 $\theta^T \phi(x) = 0$ 은 원래 공간 (x_1, x_2) 에서 보면 다음과 같습니다.

$$w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 = 0$$

이것은 원, 타원, 쌍곡선 같은 2차 곡선(Conic Section)의 방정식입니다. 즉, "고차원에서의 평면 = 저차원에서의 곡선"입니다.

1.4 4. 장점과 주의점 (The Trade-off)

□ 개념 4: 공짜 점심은 없다

한 줄 요약: 표현력(Power)이 좋아지지만, 그만큼 계산 비용과 과적합(Overfitting) 위험이 폭증합니다.

1) 차원의 저주 (Curse of Dimensionality)

차수(Degree) Q 를 높이면 특징의 개수 D 가 기하급수적으로 늘어납니다.

$$D \approx d^Q$$

- 입력 변수가 100개($d = 100$)인데 3차 다항식($Q = 3$)을 쓰면? 특징 개수가 약 100만 개가 됩니다.
- 계산 문제: 100만 차원 벡터의 내적을 계산해야 함 → 느려짐.
- 통계적 문제: 변수가 100만 개인데 데이터가 1만 개라면? $\rightarrow p \gg n$ 문제 발생 $\rightarrow$ 과적합(Overfitting).

커널 트릭 (Kernel Trick) 예고

"특징 개수가 100만 개라도, 실제로 $\phi(x)$ 를 계산하지 않고 내적값 $\phi(x) \cdot \phi(y)$ 만 빠르게 구할 수 있다면?" 이것이 다음 단원에서 배울 SVM 커널 트릭의 핵심 아이디어입니다.

2 실전 시나리오: 불법 프로그램(핵) 사용자 탐지

□ Scenario: 넥슨 FPS 게임 '서든어택'의 에임봇 탐지

당신은 보안팀입니다. 유저의 마우스 움직임 데이터를 분석해 '에임봇(Auto-aim)' 사용자를 잡고 싶습니다.

1. 데이터: x_1 (조준 속도), x_2 (정확도).
2. 분포 확인:
 - 일반 유저: 속도가 빠르면 정확도가 낮고, 느리면 높음 (반비례). 넓게 퍼져 있음.
 - 핵 유저: 속도가 엄청 빠르면서 동시에 정확도도 100%임. 특정 영역(우상단)에 몰려 있음.
3. 선형 분류 시도: 직선을 그어보지만, 고수 유저(빠르고 정확함)와 핵 유저가 섞여서 구분이 안 됩니다.
4. 다항식 특징 추가: x_1^2, x_2^2, x_1x_2 를 추가합니다.
 - 특히 $x_1 \times x_2$ (속도 $\times$ 정확도) 라는 교호작용 항(Interaction Term)이 핵심입니다.
 - 일반 유저는 이 값이 일정 한계를 넘지 못하지만(인간의 한계), 핵 유저는 이 값이 비정상적으로 높습니다.
5. 결과: 고차원 공간에서 핵 유저 그룹만 뚝 떨어져 나와, 평면 하나로 깔끔하게 검거(Classification)할 수 있게 되었습니다.

3 자주 묻는 질문 (FAQ)

- Q1. 차수는 몇 차(Degree)까지 늘려야 하나요?** A. 정답은 없습니다. 보통 2차나 3차 정도만 써도 충분한 경우가 많습니다. 너무 높으면 훈련 데이터는 다 맞추지만(Overfitting), 새로운 데이터는 다 틀리는 괴물이 탄생합니다. 교차 검증(Cross Validation)으로 적절한 차수를 찾아야 합니다.
- Q2. 요즘 딥러닝은 특징 공학을 안 한다던데요?** A. 맞습니다! 딥러닝(Neural Networks)의 가장 큰 장점은, 사람이 손으로 $\phi(x)$ 를 설계하는 대신, 네트워크가 스스로 최적의 특징 $\phi(x)$ 를 찾아낸다(Representation Learning)는 점입니다. 다음 Unit 3에서 바로 그 내용을 배웁니다.

Next Step: 우리는 사람이 직접 $\phi(x) = x^2$ 처럼 식을 정해주는 방식을 배웠습니다. 그런데 이미지나 음성처럼 복잡한 데이터는 어떤 식을 써야 할지 감도 안 옵니다. 컴퓨터가 스스로 $\phi(x)$ 를 만들게 할 수는 없을까요? 다음 **Unit 3: 신경망(Neural Networks)**에서는 퍼셉트론을 여러 겹 쌓아 스스로 특징을 학습하는 딥러닝의 기초를 배웁니다.

Unit 2 (Part B) 핵심 요약

- **문제:** 현실 데이터는 직선 하나로 나뉘지 않는다 (Non-linear).
- **해결:** 데이터를 고차원으로 매핑 ($\phi(x)$) 하면 선형 분리가 가능해진다.
- **방법:** 다항식 특징 ($1, x, x^2, \dots$) 이 가장 대표적이다.
- **대가:** 차원이 늘어나면 계산량이 폭증하고 과적합 위험이 생긴다.
- **의의:** 딥러닝 이전에는 사람이 이 특징 (ϕ) 을 찾는 것이 머신러닝의 전부였다.